

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 10
JAVA COLLECTIONS**



Disusun oleh :

Naufal Fakhrianto Nugroho 105224026

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026**

BAB I

PENDAHULUAN

Praktikum Pemrograman Berorientasi Objek pada modul ini membahas implementasi Java Collections Framework (JCF) sebagai mekanisme penyimpanan data yang fleksibel dan dinamis. Tidak seperti array konvensional yang memiliki kapasitas tetap sejak deklarasi, JCF memungkinkan manipulasi kumpulan data secara real-time tanpa batasan ukuran yang kaku.

Studi kasus yang diimplementasikan adalah Sistem Manajemen Gudang berbasis Object-Oriented Programming (OOP) murni. Program ini memanfaatkan tiga komponen utama JCF: Map (HashMap) sebagai basis data barang, Set (HashSet) untuk menyimpan kategori tanpa duplikasi, dan List (ArrayList) untuk mencatat riwayat transaksi secara kronologis. Ketiga komponen tersebut diintegrasikan untuk menghasilkan sistem inventarisasi yang handal dan terstruktur di dalam memori program.

BAB II

VARIABLE

No	Nama Variable	Tipe Data	Fungsi
1	idBarang	String	Penyimpan identitas unik setiap barang
2	namaBarang	String	Penyimpan nama barang
3	kategori	String	Penyimpan kelompok kategori barang
4	stok	int	Penyimpan jumlah kuantitas stok barang
5	databaseBarang	Map	Basis data utama dari pasangan Key
6	kategoriUnik	set	Himpunan kategori tanpa duplikasi
7	riwayatAktivitas	List	Daftar urut catatan transaksi
8	id	String	Parameter pencarian barang berdasarkan ID

9	nama	String	Parameter pencarian nama saat pendaftaran barang
10	jumlah	int	Parameter kuantitas barang di proses
11	barangBaru	Barang	Objek sementara saat instansiasi barang baru
12	barang	Barang	Referensi objek barang hasil pencarian di Map
13	gudang	SistemGudang	Objek utama sistem di main
14	stokLama	int	Variable penampung stok sebelum di tambah

BAB III

CONSTRUCTOR dan METHOD

No	Nama Method	Jenis Method	Fungsi
1	Barang()	Constructor	Inisialisasi objek barang dengan semua atribut
2	getIdBarang()	Functional	Getter, ambil nilai idBarang
3	getNamaBarang()	Functional	Getter, ambil nilai namaBarang
4	getKategori()	Functional	Getter, ambil nilai kategori
5	getStok()	Functional	Getter, ambil nilai stok saat ini
6	setStok()	Procedural	Setter, memperbarui

			nilai stok barang
7	SistemGudang()	Constructor	Inisialisasi HashMap,HashSet, ArrayList
8	tambahBarangBaru()	Procedural	Daftarkan barang baru ke Map dan kategori ke Set
9	tambahStok()	Procedural	Menambah kuantitas stok berdasarkan ID
10	kurangiStok()	Procedural	Mengurangi stok dengan berdasarkan ID dan kecukupan
11	cetakLaporan()	Procedural	Menampilkan kategori, sisa stok, dan riwayat transaksi
12	main()	Procedural	Eksekusi utama simulasi sistem gudang

BAB IV

DOKUMENTASI dan PEMBAHASAN

A. Class Barang

```

1  ✓ public class Barang {
2      private String idBarang;
3      private String namaBarang;
4      private String kategori;
5      private int stok;
6
7  ✓  public Barang(String idBarang, String namaBarang, String kategori, int
8      this.idBarang = idBarang;
9      this.namaBarang = namaBarang;
10     this.kategori = kategori;
11     this.stok = stok;
12 }
13
14 ✓  public String getIdBarang(){
15     return idBarang;}
16
17 ✓  public String getNamaBarang(){
18     return namaBarang; }
19
20 ✓  public String getKategori(){
21     return kategori;}
22
23 ✓  public int getStok(){
24     return stok;}

```

Kelas Barang berfungsi sebagai kelas model (Data Object) yang merepresentasikan entitas barang fisik. Seluruh atribut dideklarasikan private untuk menerapkan enkapsulasi data. Akses terhadap nilai atribut dilakukan melalui metode getter, sedangkan perubahan nilai stok dikontrol melalui setter setStok(). Pemisahan ini memastikan tidak ada kelas eksternal yang dapat memodifikasi data barang secara langsung tanpa melewati prosedur resmi.

- Line 2 sampai 5 = Atribut idBarang, namaBarang, kategori, dan stok dideklarasikan private. Ini adalah inti dari Data Hiding pada OOP, memproteksi state internal objek dari akses ilegal.
- Line 7 sampai 12 = Konstruktor menerima empat parameter dan menggunakannya untuk menginisialisasi atribut via kata kunci this, yang membedakan parameter lokal dari atribut kelas yang bernama sama.
- Line 14 sampai 28 = Getter dan setter disediakan sebagai antarmuka akses resmi. Variabel stokLama digunakan di SistemGudang untuk menyimpan nilai stok sebelum mutasi, memastikan kalkulasi penambahan stok akurat.

B. Class Sistem Gudang

```

src > SistemGudang.java > SistemGudang > kurangStok(String, int)
1  import java.util.HashMap;
2  import java.util.HashSet;
3  import java.util.ArrayList;
4  import java.util.Map;
5  import java.util.Set;
6  import java.util.List;
7
8  public class SistemGudang {
9      private Map<String, Barang> databaseBarang;
10     private Set<String> kategoriUnik;
11     private List<String> riwayatAktivitas;
12
13     public SistemGudang() {
14         databaseBarang = new HashMap<>();
15         kategoriUnik = new HashSet<>();
16         riwayatAktivitas = new ArrayList<>();
17     }
18     public void tambahBarangBaru(String id, String nama, String kategori, int stok) {
19         if (databaseBarang.containsKey(id)) {
20             String log = "[GAGAL] Daftar Barang: ID '" + id + "' sudah ada di sistem.";
21             riwayatAktivitas.add(log);
22             System.out.println(log);
23             return;
24         }
25         Barang barangBaru = new Barang(id, nama, kategori, stok);
26         databaseBarang.put(id, barangBaru);
27         kategoriUnik.add(kategori);
28         String log = "(BARU) Barang Baru Terdaftar: " + nama + " (ID: " + id + ", Kategori: " + kategori + ", Stok Awal: " + stok + " unit)";
29         riwayatAktivitas.add(log);
30         System.out.println(log);
31     }
32     public void tambahStok(String id, int jumlah) {
33         Barang barang = databaseBarang.get(id);
34         if (barang != null) {
35             int stokLama = barang.getStok();
36             barang.setStok(stokLama + jumlah);
37             String log = "(MASUK) Tambah Stok: " + barang.getNamaBarang() + " (ID: " + id + ") + " + jumlah + " unit -> Total: " + barang.getStok() + " unit";
38             riwayatAktivitas.add(log);
39             System.out.println(log);
40         } else {
41             String log = "[GAGAL] Tambah Stok: ID '" + id + "' tidak ditemukan.";
42             riwayatAktivitas.add(log);
43             System.out.println(log);
44         }
45     }
46     public void kurangiStok(String id, int jumlah) {
47         Barang barang = databaseBarang.get(id);
48         if (barang == null) {
49             String log = "[GAGAL] Kurangi Stok: ID '" + id + "' tidak ditemukan.";
50             riwayatAktivitas.add(log);
51             System.out.println(log);
52             return;
53         }
54         if (barang.getStok() >= jumlah) {
55             barang.setStok(barang.getStok() - jumlah);
56             String log = "[OK] Kurangi Stok: " + barang.getNamaBarang() + " (ID: " + id + ") - " + jumlah + " unit -> Sisa: " + barang.getStok() + " unit";
57             riwayatAktivitas.add(log);
58             System.out.println(log);
59         } else {
60             String log = "[GAGAL] Kurangi Stok: " + barang.getNamaBarang() + " (ID: " + id + ") stok tidak mencukupi. Diminta: " + jumlah + " unit, Tersedia: " + barang.getStok();
61             riwayatAktivitas.add(log);
62             System.out.println(log);
63         }
64     }
65     public void cetakLaporan() {
66         System.out.println("LAPORAN AKHIR SISTEM GUDANG");
67
68         System.out.println("\nDAFTAR KATEGORI UNIK");
69         for (String kat : kategoriUnik) {
70             System.out.println(" * " + kat);
71         }
72
73         System.out.println("\nSISA STOK SELURUH BARANG DI GUDANG");
74         System.out.printf(format: " %-6s %-15s %-12s %s\n", ...args: "ID", "Nama", "Kategori", "Stok");
75         System.out.println(" " + "-".repeat(count: 45));
76         for (Barang b : databaseBarang.values()) {
77             System.out.printf(format: " %-6s %-15s %-12s %d unit\n", b.getIdBarang(), b.getNamaBarang(), b.getKategori(), b.getStok());
78         }
79
80         System.out.println("\nRIWAYAT AKTIVITAS TRANSAKSI");
81         int urutan = 1;
82         for (String riwayat : riwayatAktivitas) {
83             System.out.println(" " + urutan + ". " + riwayat);
84             urutan++;
85         }
86     }
87 }
88

```

Kelas SistemGudang berperan sebagai controller yang mengelola seluruh logika bisnis mutasi inventaris. Atribut dideklarasikan menggunakan tipe interface (Map, Set, List) dan diinstansiasi dengan kelas konkret di dalam konstruktor

- tambahBarangBaru() = Validasi keunikan ID melalui containsKey(). Jika lolos, objek Barang dimasukkan ke HashMap via put(), kategori dimasukkan ke HashSet via add(), dan log sukses dicatat ke ArrayList.
- tambahStok() = Pencarian O(1) menggunakan get(id). Jika ditemukan, stok baru dihitung dari stokLama + jumlah dan disimpan kembali via setStok().
- kurangiStok() = Pengkondisian berlapis validasi keberadaan ID, lalu validasi kecukupan stok via getStok() >= jumlah. Transaksi yang gagal langsung dicetak ke konsol dan dicatat ke riwayat.
- cetakLaporan() = Iterasi For-Each pada Set, Map.values(), dan List menghasilkan ringkasan kategori unik, tabel stok akhir, dan kronologi transaksi bernomorurut.

C. Class Main

```
src > Main.java > Main > main(String[])
1  public class Main {
2      public static void main(String[] args) {
3          SistemGudang gudang = new SistemGudang();
4
5          System.out.println(x: " SIMULASI SISTEM MANAJEMEN GUDANG \n");
6          gudang.tambahBarangBaru(id: "G505", nama: "HandPhone", kategori: "Elektronik", stok: 8);
7          gudang.tambahBarangBaru(id: "G122", nama: "LemariBaju", kategori: "Furnitur", stok: 12);
8          gudang.tambahBarangBaru(id: "G696", nama: "Laptop", kategori: "Elektronik", stok: 20);
9
10         System.out.println();
11         gudang.tambahStok(id: "G505", jumlah: 7);
12
13         System.out.println();
14         gudang.kurangiStok(id: "G696", jumlah: 5);
15
16         System.out.println();
17         gudang.kurangiStok(id: "G122", jumlah: 50);
18
19         gudang.cetakLaporan();
20     }
21 }
```

Kelas Main merupakan titik eksekusi program. Objek SistemGudang diinstansiasi di baris pertama, yang secara otomatis menginisialisasi ketiga struktur data JCF di dalam memori.

- Line 3 = SistemGudang gudang = new SistemGudang(); ini merupakan instansiasi objek utama gudang. Proses ini akan memicu method SistemGudang untuk alokasi ruang kosong bagi data HashMap, HashSet, dan ArrayList
- Line 5 sampai 8 = Tiga barang didaftarkan (HandPhone, Lemari Baju, Laptop). HashSet secara otomatis menyaring duplikasi meskipun kategori 'Elektronik' dimasukkan dua kali, Set hanya menyimpan satu entri unik.
- Line 10 sampai 11 = tambahStok("G505", 7) berhasil menambah stok HandPhone dari 8 menjadi 15 unit. Variabel stokLama digunakan untuk kalkulasi yang aman.

- Line 13 sampai 14 = `kurangiStok("G696", 5)` berhasil mengurangi stok Laptop dari 20 menjadi 15 unit karena kondisi `getStok() >=` jumlah terpenuhi.
- Line 16 sampai 17 = `kurangiStok("G122", 50)` gagal karena Lemari Baju hanya tersedia 12 unit sementara permintaan 50 unit. Sistem menolak transaksi, mencetak peringatan ke konsol, dan mencatat log kegagalan ke riwayat tanpa mengubah stok asli.
- Line 19 = Method `cetakLaporan()` dipanggil dan menampilkan data barang dari daftar kategori, hingga penambahan atau mengurangi barang pada stok asli

BAB V

KESIMPULAN

Berdasarkan implementasi program simulasi Sistem Manajemen Gudang, dapat disimpulkan bahwa Java Collections Framework menyediakan solusi penyimpanan data yang adaptif dan efisien. Tiga komponen utama JCF `HashMap`, `HashSet`, dan `ArrayList` terbukti mampu memenuhi kebutuhan fungsional sistem inventarisasi secara optimal.

`HashMap` sebagai basis data utama menjamin kecepatan pencarian $O(1)$ berbasis kunci ID, sehingga operasi verifikasi dan mutasi stok berjalan sangat cepat. `HashSet` secara otomatis mengelola keunikan kategori tanpa membutuhkan pengecekan manual. `ArrayList` merekam seluruh jejak transaksi secara kronologis dengan presisi urutan.

Perbedaan signifikan dibandingkan pendekatan array konvensional terletak pada fleksibilitas ukuran dinamis dan ketersediaan API manipulasi data bawaan. Kombinasi JCF dengan prinsip enkapsulasi OOP menghasilkan sistem yang aman, terstruktur, dan mudah diperluas.

BAB VI

DAFTAR PUSTAKA

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemrograman Berorientasi Objek: Java Collections*. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.